

From Zero to Julia: Key Points Extraction

Source: <https://techtok.com/from-zero-to-julia/>

VSCode Setup

Installing Julia

- Download from julialang.org/downloads/ — install the latest stable version.
- **Windows/macOS:** Run the installer; choose your home directory as the install location.
- **Linux:** Download and extract the binary archive (match your architecture) into an appropriate folder (e.g., home directory).
- **Add Julia to your PATH:**
- Windows: check the "add to PATH" option during installation.
- macOS/Linux: add to `.bashrc` or `.zshrc`:

```
bash export PATH="$PATH:/path/to/<Julia directory>/bin"
```

Setting Up VSCode

- Install VSCode from code.visualstudio.com.
- Install the **"Julia" extension** from the Extensions panel (search for `Julia`).
- In the extension settings, set `Julia: Executable Path` to the path of your Julia binary.
- Optionally set `Execution: Result Type` to `inline` to see results next to each line (notebook-like experience).

Running Code

- Julia files use the `.jl` extension.
- **Run a single line:** Place cursor on the line and press `Shift+Enter`. The code is sent to the **REPL** (Julia interactive prompt) which opens in the bottom panel.
- First launch may take a while (Language Server loading). If you get errors, wait and retry.

Code Cells

- Enclose blocks of code between `%%` markers to define cells:

```
julia %% this is a code cell println("Hello") println("This is TechyTok!") %%
```

- **Run a cell:** Place cursor inside the cell and press `Alt+Enter`.

- You can add comments after `###` to label each cell (since `#` starts a comment in Julia).
- Best practice: write functions at the top of the file, usage at the bottom, and annotate each cell with a brief description.

Key Takeaways

- VSCode + Julia extension is the **recommended IDE** (Juno/Atom is in maintenance mode).
 - Core shortcuts: `Shift+Enter` (run line), `Alt+Enter` (run cell).
 - The REPL panel at the bottom is where your code actually executes.
-

Introduction to the REPL

Introduction to the REPL

Two ways to run Julia code:

1. **Julia REPL** (interactive prompt)
2. **VSCode** (for scripts and longer code)

REPL Basics

- Launch the Julia executable to open the REPL (Read-Eval-Print Loop).
- Type Julia expressions directly and press Enter to execute them.
- Example:

```
julia print("Hello World!")
```
- Best for quick experiments and learning.

VSCode for Scripts

- Julia scripts use the `.jl` file extension.
- **Run a single line:** place cursor on the line → `Alt+Enter`
- **Run a code block:** wrap code between `###` markers → `Alt+Shift+Enter`

```
julia ### println("hello") println("from") println("TechyTok!") ###
```

Key Takeaway

- Use the **REPL** for quick, interactive testing.
 - Use **VSCode** when code grows longer and needs to be organized into `.jl` files.
-

Lesson 1: Variables and Types

Variables

- Variables are assigned with `=` and Julia **infers the type automatically** (no type declaration needed, unlike C++).

```
julia my_name = "Aurelio" # String my_favourite_number = 42 # Int64
my_favourite_pie = 3.1415 # Float64
```

- Print a variable's value with `print(my_name)`.

Basic Math Operations

```
a = 2; b = 3
sum      = a + b
difference = a - b
product  = a * b
quotient = b / a
power    = a^3
modulus  = b % a
```

- Rational numbers** for exact arithmetic use `//` syntax: `1//3`.

Types

- Every value in Julia has a type. The type system is **hierarchical**: `Any` $\rightarrow$ `Number` $\rightarrow$ `Real` $\rightarrow$ `Int64` / `Float64`, etc.
- Check a type with `typeof()`:

```
julia typeof(0.1) # Float64 typeof(42) # Int64 typeof("hello") # String
```
- Because Julia knows the types, it can compile **optimized machine code** — this is a key reason Julia is fast.

Type Stability (Performance Best Practice)

- Keep a variable's type **consistent** throughout its lifetime. Avoid reassigning `a = 42` to `a = 0.42`.
- Initialize with the correct type from the start:

```
julia a = 2 # use if you need Int b = 2.0 # use if you need Float64
```

Type Conversion

- Use `convert(TargetType, value)` to explicitly change a type:

```
julia convert(Float64, 2) # → 2.0
```
-

Lesson 2: Functions

Functions are the fundamental building blocks in Julia. Even operators like `+` are functions in infix form.

Defining Functions

Three ways to define a function:

- **Standard form:**

```
julia function plus_two(x) return x + 2 end
```

- Starts with `function`, ends with `end`.
- Body must be indented (convention: 4 spaces).

- **Inline (one-liner) form:**

```
julia plus_two(x) = x + 2
```

- **Anonymous function (lambda-style):**

```
julia plus_two = x -> x + 2
```

- Use sparingly — only for very simple cases (e.g., passing a quick wrapper to another function).

Return Values

- Use `return` to explicitly return a value.
- If `return` is omitted, Julia returns the **last evaluated expression**.
- Void functions (no return value) can omit `return` entirely.

Nested Functions

- Functions can be defined **inside other functions**.
- Useful for creating small helper/alias functions to improve readability.
- Inner function names won't conflict with outer scope.

Optional Positional Arguments

```
function myWeight(weightOnEarth, g=9.81)
    return weightOnEarth * g / 9.81
end
```

- Default values make arguments optional.
- Order matters — you **cannot** reorder positional arguments by name (unlike Python).

Keyword Arguments

- Separated from positional args by a **semicolon** `;`.
- Must be called **by name**; order doesn't matter.
- Can be required (no default) or optional (with default).

```
function my_long_function(a, b=2; c, d=3)
    return a + b + c + d
end

my_long_function(1, c=3)           # → 9
my_long_function(1, 2, d=5, c=3)  # → 11
```

Performance tip: Prefer positional arguments in hot loops / performance-critical code. Use keyword arguments for convenience in higher-level wrappers.

Documentation

- Access docs in the REPL with `? functionName`.
- Write your own docs with a triple-quoted string above the function:

```
julia """ Description of the function """ function foo(x) # ... end
```

Lesson 3: Data Structures

Arrays

Vectors (1D Arrays)

- Created with square brackets: `a = [1, 2, 3, 4, 5]`
- **Julia is 1-indexed** — first element is `a[1]`, not `a[0]`
- Elements must share a common type. Appending a wrong type throws an error.
- Append with `append!(a, 6)` — the `!` convention means the function mutates its argument.
- Check type with `typeof(a) → Array{Int64, 1}`
- Explicit type declaration: `d = Int[1, 2, 3, 4, 5]`

Matrices (2D Arrays)

- Rows separated by `;`, columns by spaces:

```
julia mat1 = [1 2 3; 4 5 6]
```

- Access elements: `mat1[row, col]`

N-dimensional Arrays

- No literal syntax; use `zeros(dims...)` then fill with loops:

```
julia table = zeros(2, 3, 4)
```
- **Performance tip:** The first index should vary fastest (column-major order). Store 42 matrices of size 2×2 as `zeros(2, 2, 42)`, not `zeros(42, 2, 2)`.

Slicing

- Extract a range of elements: `a[2:5]`
- Works on matrices too: `mat1[2:3, 2:3]`
- **List comprehension:** `[i for i in 1:16]` creates an array from 1 to 16.
- Nested comprehension: `[i+j for i in 1:10 for j in 1:5]`

Views vs. Copies

- Assignment `b = a` creates a **reference** (pointer), not a copy. Modifying `b` also modifies `a`.
 - Use `copy(a)` for a shallow copy, `deepcopy(a)` for nested arrays.
-

Tuples

- Fixed-size, immutable group of values (may have mixed types).
- Created with or without parentheses: `a = (1, 2, 3)` or `a = 1, 2, 3`
- **Unpacking:** `a, b, c = tuple1`
- Useful for multiple return values from functions:

```
julia function return_multiple() return 42, 43, 44 end a, b, c = return_multiple()
```
- **Splatting** `(...)` unpacks a tuple as function arguments: `splat_me(tuple1...)`

Named Tuples

- Associate names with values:

```
julia nt = (a = 1, b = "hello") nt[:a] # → 1
```
-

Dictionaries

- Unordered key-value pairs. Created with `Dict`:

```
julia person = Dict("Name" => "Aurelio", "Phone" => 123456789)
```
- Add entries after creation: `addressBook["Vittorio"] = person3`
- Dictionaries can be nested (e.g., a dict of dicts).

General Notes

- Julia has a built-in garbage collector — no manual memory management needed.
- The `!` suffix on function names (e.g., `append!`) signals that the function mutates its input.

Lesson 4: Control Flow

If / Elseif / Else

- Conditional blocks are closed with `end` (same as functions).
- Use `elseif` for multiple conditions (checked in order, first match wins).

```
if x >= 0
    return x
elseif x < 3
    return -x
else
    return 0
end
```

- Combine conditions with `&` (AND) and `||` (OR):

```
if 1 < 3 & 3 < 4
    print("both true")
end
```

String Interpolation

- Use `$variable` inside a string to insert its value:

```
name = "Alice"
print("Hello $name!") # → Hello Alice!
```

For Loops

- Iterate over a range or any iterable collection:

```
for i in 1:10
    println(i^2)
end
```

- `for i = 1:10` is equivalent syntax.
- You can loop over arrays directly:

```
for person in ["Alice", "Bob"]
    println("Hello $person")
end
```

Break and Continue

- `break` — exits the loop immediately.
- `continue` — skips the current iteration and moves to the next.

```
for i in 1:30
    if i % 3 == 0
        continue # skip multiples of 3
    end
    println(i)
end
```

While Loops

- Repeats as long as a condition is true. You must manually update the loop variable.

```
i = 0
while i < 30
    println(i)
    i += 1
end
```

Enumerate

- `enumerate(collection)` yields `(index, value)` pairs — useful when you need both the position and the element.

```
x = ["a", "b", "c"]
for (i, val) in enumerate(x)
    println("$i: $val")
end
# 1: a
# 2: b
# 3: c
```

- Common pattern: filling a results array by index:


```
result = zeros(length(input))
for (i, elem) in enumerate(input)
    result[i] = elem^2
end
```

Lesson 5: Working with Arrays (Broadcasting)

Core Concept

Broadcasting applies a function or operation **element-by-element** over an array. It is purely a syntactic convenience — there is **no performance gain** over a `for` loop.

Why It Exists

Unlike Python/Matlab, Julia's `for` loops are already as fast as C. There is no need to write "vectorised" code for speed. Broadcasting is used for **conciseness**, not performance.

Syntax

- **Operators:** add a `.` before the operator → `.+`, `.-`, `.*`, `./`, `.^`
- **Functions:** add a `.` before the parentheses → `sin.(x)`, `exp.(x)`

Examples

```
a = [1, 2, 3]
b = [4, 5, 6]

# Element-wise multiplication
a .* b          # → [4, 10, 18]

# Apply a function element-wise
sin.(a)         # → [0.841, 0.909, 0.141]
```

Default (Non-Broadcast) Behaviour

Without the dot, operators follow **linear algebra rules**:

```
a = [1, 2, 3]    # column vector
c = [4 5 6]      # row vector

a * c            # → 3×3 matrix (outer product)
c * a            # → scalar (dot product = 32)
```

Key Takeaways

- **Write functions that accept single values**, then broadcast them when needed.
- Broadcasting is equivalent in speed to a `for` loop — use it for readability.
- Do **not** force vectorised code the way you would in Python/NumPy; it makes Julia code harder to read and offers no speed benefit.
- `map(f, arr)` also works but broadcasting syntax (`f.(arr)`) is generally preferred for its flexibility.
- For parallelism, `for` loops and `map/pmap` are the natural entry points.

Lesson 6: Variable Scope

Scope = the region of a program where a variable is accessible.

Two Types of Scope

- **Global scope:** Variable is accessible everywhere. Created when you define a variable in the REPL or outside any function/construct.
- **Local scope:** Variable is only accessible within that scope (and nested scopes inside it).

Constructs That Introduce Scope

Construct	Scope Type
<code>module</code> , <code>baremodule</code> , REPL	Global
<code>struct</code> , <code>macro</code>	Local
<code>for</code> , <code>while</code> , <code>try-catch-finally</code> , <code>let</code>	Local
Functions (all forms), comprehensions	Local

Local Scope in Functions

Variables defined inside a function are **not** accessible outside it:

```
function example()  
    z = 42  
end  
z # ERROR: UndefVarError: z not defined
```

- **Best practice:** Return values from functions instead of modifying globals.
- You can use `global z = 42` inside a function to write to the global scope, but this is

discouraged.

- Use `local` keyword to explicitly shadow a global variable inside a local scope.

`let` Blocks

Introduce a new local scope. Useful for isolating intermediate computations:

```
a = let
  i = 3
  i += 5
  i # last expression is the return value
end
# a == 8, but i is not defined outside
```

Key difference: `begin` blocks do **not** introduce a new scope — variables leak out.

```
d = begin
  i = 41
  i += 1
  i
end
# d == 42 AND i == 42 (i leaks into outer scope)
```

Global Scope & Performance

- **Avoid global variables** — they hurt performance because the compiler can't optimize them (type may change at runtime).
- If you must use globals, declare them as **constants**:

```
const C = 299792458 # type is fixed as Int
C = 300000000 # WARNING (value change allowed but discouraged)
C = 2.998e8 # ERROR (type change not allowed)
```

`const` guarantees the **type** won't change, enabling compiler optimizations.

Modules and Scope

Each `module` has its own global scope. Variables inside a module are not accessible outside unless:

- They are `exported`, or
- Accessed via `ModuleName.varName`

You **cannot** assign to variables in other modules.

Best Practices Summary

1. Avoid global variables; if needed, use `const`.
 2. Pass all needed data as function arguments; functions should operate only on their inputs.
 3. Use functions and `let` blocks to create local scopes and avoid name collisions.
-

Lesson 7: Modules

Lesson 7: Modules — Key Points

What Are Modules?

- A module is a **separate scope** containing its own variables and functions.
- Some functions/variables are **exported** (accessible by name directly); others require the `ModuleName.functionName` syntax.
- Modules are **not** like classes in OOP — you cannot instantiate them multiple times. Functions inside a module should depend only on their inputs.

Installing & Importing Packages

```
using Pkg
Pkg.add("SpecialFunctions")    # install a package

using SpecialFunctions         # import all exported functions
using SpecialFunctions: gamma, sinint # import only specific functions
import SpecialFunctions        # import module WITHOUT exporting anything into scope
```

- `using ModuleName` → brings exported names into current scope.
- `using ModuleName: func1, func2` → imports only selected functions.
- `import ModuleName` → loads the module but requires fully qualified calls (`ModuleName.func()`). Useful to avoid name conflicts.

Resolving Name Conflicts

- If a local function has the same name as an exported one, the local definition takes precedence.
- Use `ModuleName.functionName()` to explicitly call the module's version.
- Prefer `import` over `using` when conflicts are likely.

Defining Your Own Module

```
module MyModule
export func2

a = 42

function func1(x)
    return x^2
end

function func2(x)
    return func1(x) + a
end
end
```

- Use `module Name ... end` (no indentation inside the block).
- `export` declares which names are publicly accessible.
- Non-exported functions are accessed via `MyModule.func1(3)`.
- Import a local module with a dot: `using .MyModule` (the `.` means it's defined in `Main`).

Splitting Code Across Files with `include()`

- `include("path/to/file.jl")` pastes the file's content into the current scope.
- Recommended structure:
- One **main module file** that calls `include(...)` for each sub-file and declares `export`.
- Multiple **sub-files** grouping related functions.

```
# big-module.jl
module MyBigModule
include("files/big-module-part-1.jl")
include("files/big-module-part-2.jl")
export func2big
end
```

- Include files in logical/dependency order (good practice, though not strictly required by Julia).
- Do **not** nest `include()` calls inside files that are themselves included.

Documenting Functions

```
@doc raw"""
    func2big(x)

Computes the square of `x` and adds `a` to it.
```

```
# Examples
```julia-repl
julia> func2big(3)
51
```

```
"""
```

```
function func2big(x)
return func1big(x) + a
end
```

- Place a docstring (triple-quoted) directly above the function.
- Use ``@doc raw"""..."""`` to allow markdown without escaping.
- Convention: start with the function signature (indented 4 spaces), then a summary,
- Access docs in the REPL with ``? functionName``.

```
Best Practices Summary
```

- Give functions **meaningful, general names** for reusability.
- Keep functions **stand-alone** (depend on inputs, not hidden state).
- **Document** every public function.
- **Split** large modules into multiple files for maintainability.

```

```

```
Lesson 8: Types and Structures
```

```
Core Concepts
```

- **Types** are containers for data. Julia has a type hierarchy: parent types are abstract, child types are concrete.
- **Abstract types** cannot be instantiated; they serve as categories. **Concrete types** can be instantiated.
- **Immutable structs** (``struct``) – fields cannot be changed after creation.
- **Mutable structs** (``mutable struct``) – fields can be modified after creation.

```
Defining Types
```

```
```julia
```

```
# Abstract type with subtype relationship using <:
abstract type Person end
abstract type Musician <: Person end
```

```
# Immutable concrete type
struct ClassicMusician <: Musician
    name::String
    instrument::String
end
```

```
# Mutable concrete type
mutable struct Rockstar <: Musician
    name::String
    instrument::String
    bandName::String
    headbandColor::String
    instrumentsPlayed::Int
end
```

Instantiation & Mutability

```
aure = ClassicMusician("Aurelio", "Violin")
aure.instrument = "Cello" # ERROR: immutable struct cannot be changed

ricky = Rockstar("Riccardo", "Voice", "Black Lotus", "red", 2)
ricky.headbandColor = "blue" # Works fine (mutable)
```

Multiple Dispatch

- You can define multiple methods for the same function name with different type signatures.
- Julia calls the method whose type signature is **most specific** (lowest in the type tree).

```
function introduceMe(person::Person)
    println("Hello, my name is $(person.name).")
end

function introduceMe(person::Musician)
    println("Hello, my name is $(person.name) and I play $(person.instrument).")
end
```

If you pass a `Rockstar`, Julia picks the `Rockstar` method (if defined), then `Musician`, then `Person` — whichever is most specific.

Type Constructors

- Default constructors are auto-generated when you define a struct.
- You can define **inner constructors** (inside the struct body) for custom logic, default values, or validation.

```
mutable struct MyData
    x::Float64
    x2::Float64
    y::Float64
    z::Float64
    function MyData(x::Float64, y::Float64)
        x2 = x^2
        z = sin(x2 + y)
        new(x, x2, y, z)
    end
end
```

Parametric Types

- Use parametric types to keep flexibility while ensuring concrete types inside the struct

(important for performance).

```
mutable struct MyData2{T<:Real}
  x::T
  x2::T
  y::T
  z::Float64
  function MyData2{T}(x::T, y::T) where {T<:Real}
    x2 = x^2
    z = sin(x2 + y)
    new(x, x2, y, z)
  end
end

MyData2{Float64}(2.0, 3.0) # All fields are Float64
MyData2{Int}(2, 3)        # x, x2, y are Int; z is still Float64
```

Practical Pattern: Mutable Structs for Shared State

- Mutable structs are useful for storing computed results that multiple functions in a module need to share.

```
mutable struct Circle{T<:Real}
  radius::T
  perimeter::Float64
  area::Float64
  function Circle{T}(radius::T) where T<:Real
    new(radius, -1.0, -1.0) # -1.0 as "not yet computed" sentinel
  end
end

function computeArea(circle::Circle)
  circle.area = π * circle.radius^2
  return circle.area
end
```

Key Takeaways

Concept	Keyword / Syntax
Abstract type	<code>abstract type Name end</code>
Subtype	<code><:</code>
Immutable struct	<code>struct Name ... end</code>
Mutable struct	<code>mutable struct Name ... end</code>
Type annotation	<code>arg::Type</code>

Parametric type	<code>struct Name{T<:Real} ... end</code>
Inner constructor	<code>function Name(...) ... new(...) end</code> inside struct body

Lesson 9: Working with Packages

Lesson 9: Working with Packages

What is a Julia package?

- A package = a module + dependency information.
- Advantages over a plain module: precompilation (faster loading) and automatic dependency management.

Creating a package

- Enter the package manager by typing `]` in the REPL, then:

```
generate TestPackage1
```

- This creates a directory structure with a skeleton module at `TestPackage1/src/`

```
TestPackage1.jl:
```

```
julia module TestPackage1 greet() = print("Hello World!") end
```

Using a user-defined package

- Set your working directory to the parent folder of the package (use `pwd()` to check, `cd(...)` to change).

- Activate and use it:

```
``julia
```

```
using Pkg
```

```
Pkg.activate("TestPackage1")
```

```
using TestPackage1
```

```
TestPackage1.greet()
```

```
` - No.` prefix is needed (unlike local modules) because it's recognized as a proper package.
```

Adding dependencies

- In the package manager (`]`), type:

```
add SpecialFunctions
```

- Then use the dependency inside your module normally:

```
``julia
```

```
module TestPackage1
```

```
using SpecialFunctions
```

```
export mySpecialFunction
```

```
function mySpecialFunction(x)
```

```
return x^2 * gamma(x)
```

```
end
```

```
end
```

```
``
```

Important: Restart the REPL after modifying package source code for changes to take effect.

Installing dependencies on a new machine

- Activate the package, then run:

```
julia using Pkg Pkg.instantiate()
```

This reads `Project.toml` / `Manifest.toml` and installs all required dependencies automatically.

Lesson 10: Coding Workflow with VSCode

Revise (Hot-Reload for Packages)

- **Revise** lets you modify package code without restarting Julia or recompiling.
 - Install: `using Pkg; Pkg.add("Revise")`
 - **Must be loaded before your package** to work:

```
julia using Pkg Pkg.activate("MyPackage") using Revise using MyPackage
```
 - After editing and saving a file, changes are reflected immediately in the REPL — no recompilation needed.
-

Code Formatting

- Press `ALT+SHIFT+Enter` to auto-format the current file (or a selected block).
 - Requires the Julia Language Server to be running (execute some code first).
-

Multi-Panel Coding

- **Split up/down/left/right:** Right-click a file tab → opens the same file in a side-by-side panel. Edits are mirrored.
 - **Split in Group:** Splits the same file within one panel — useful for viewing two distant parts of the same file simultaneously.
-

Code Navigation

- `CTRL + hover` over a function name → shows its signature in a tooltip.
 - `CTRL + Click` → jumps to the function definition.
 - **Limitation:** Only works within the same module/namespace (not from scripts into module internals).
-

Find and Replace

- `CTRL+F` → find in current file.
 - `CTRL+SHIFT+F` → find/replace across the entire project.
 - Right-click a folder → "Find in Folder" for scoped search.
-

Multi-Line Cursor

- Hold `ALT` and click multiple positions to place cursors simultaneously.
 - Typing edits all cursor positions at once — great for quick repetitive renames.
-

Documentation Panel

- Located in the Julia extension sidebar.
 - Renders docstrings as Markdown (like official docs).
 - Works for your own functions if you document them with docstrings.
-

Variable Explorer (Workspace)

- Found under the Julia extension panel → "WORKSPACE".
 - Shows all defined variables per namespace/module.
 - Click the grid icon next to a matrix/array to view it in a table view.
 - Also lists functions defined in a module.
-

Debugging

Starting the debugger:

- Debug a whole script: press `F5` or click the debug icon.
- Debug a single function interactively:

```
julia @run my_function(args) # runs until first breakpoint @enter my_function(args)
# pauses at the first line inside the function
```

Breakpoints:

- Click to the left of a line number to add a red breakpoint.
- Manage them in the "Breakpoints" panel (enable/disable without removing).

Navigation controls:

Action	Shortcut
Continue (to next breakpoint)	<code>F5</code>
Step Over (next line)	<code>F10</code>

| Step Into (enter function call) | `F11` |

| Step Out (return from function) | `Shift+F11` |

Debugger panels:

- **Variables:** Shows current variable values; double-click to edit a variable mid-run.
 - **Watch:** Monitor arbitrary expressions (e.g., `sum(a.^2)`).
 - **Call Stack:** Shows the chain of function calls.
 - **Debug Console:** A REPL inside the function's namespace — evaluate expressions or mutate variables on the fly.
-

Function Profiling

- Use `@time` / `@btime` for overall timing; use **profiling** to find per-line bottlenecks.
- REPL-based profiling:

```
julia using Profile my_func() # warm up (compile) Profile.clear() @profile  
my_func() Profile.print()
```

- **VSCode visual profiling** (preferred):

```
julia my_func() # warm up @profview my_func()
```

Opens a flame graph / list view showing which functions consume the most time.

Problem Tab

- VSCode's "Problems" tab flags potential issues (wrong argument counts, redefined constants, etc.) without running the code.
-

TODO Explorer

- Write inline annotations as comments:

```
julia # TODO: add documentation # FIXME: wrong argument passed
```

- Install the **Todo Tree** VSCode extension to color-code and list all TODOs/FIXMEs in a sidebar tree.
-

Lesson 11: Introduction to Plotting

What is `Plots.jl`?

- `Plots.jl` is a **wrapper package** that provides a unified API over multiple plotting backends (GR, Plotly, PyPlot, etc.).
- You write code once and can switch backends easily.

Installation & Setup

```
using Pkg
Pkg.add("Plots")
using Plots
```

- Default backend is **GR**.
- Switch backends by calling their name as a function: `plotly()`, `pyplot()`, `gr()`.

Basic Plotting

```
x = 1:0.01:10*pi
y = sin.(x)           # broadcasting: apply sin to each element

plot(x, y, label="sin(x)")
plot!(xlab="x", ylab="f(x)")
```

- `plot(x, y, ...)` creates a new plot.
- `plot!(...)` **modifies** the current plot (the `!` convention means mutation).
- Use `label=` to name a series (appears in the legend).

Adding Curves & Styling

```
plot!(x, y2, label="sin(x)^2", color=:red, line=:dash)
```

- Common attributes: `color`, `line` (style), `label`, `xlab`, `ylab`, `legend`.

Axis & Saving

```
xaxis!(:log10)           # set x-axis to log scale
savefig("output.png")    # save current plot to file
```

Backend Comparison

Backend	Best For	Notes
GR	Fast, default plotting	Good general-purpose choice
Plotly	Interactive plots	Outputs HTML; embeddable in web pages
PyPlot	Advanced customization	Requires Python; supports LaTeX labels

LaTeX Labels (with PyPlot backend)

```
using LaTeXStrings
plot(x, y, label=L"\sin(x)^2$")
```

- Prefix string with `L` and wrap LaTeX code in `$...$`.

Key Takeaways

- Use **broadcasting** (`.( )`) to apply functions element-wise before plotting.
- Use `plot!` to add curves, change labels, or modify an existing plot.
- Choose your backend based on need: speed (GR), interactivity (Plotly), or customization (PyPlot).

Lesson 12: Numerical Integration

Package: `QuadGK` — performs adaptive Gauss-Kronrod quadrature for 1D integrals.

Installation:

```
using Pkg
Pkg.add("QuadGK")
```

Basic Usage

- The main function is `quadgk(func, lower_bound, upper_bound)`.
- It returns a tuple: `(result, estimated_error)`.
- Supports infinite bounds (`-Inf, Inf`).

```
using QuadGK

func1(x) = exp(-x^2)
res, err = quadgk(func1, -Inf, Inf)
# res ≈ sqrt(π)
```

Controlling Accuracy

- **rtol** (relative tolerance): Integration refines until successive estimates differ by less than `rtol`. Lower = more precise but slower.

```
julia res, err = quadgk(func1, -Inf, Inf, rtol=1e-15)
```

- **order**: Controls the quadrature rule complexity (default = 7). Higher order can

integrate higher-degree polynomials exactly. Increase if convergence is difficult, but expect longer computation time.

```
julia res, err = quadgk(func1, -Inf, Inf, order=12)
```

Integrating Functions with Multiple Arguments

When your function has extra parameters, wrap it so only the integration variable remains:

Option 1 — Named wrapper function:

```
func2(x, y, z) = x + y^3 + sin(z)

x = 5; z = 3
arg(y) = func2(x, y, z)
quadgk(arg, 1, 3)
```

Option 2 — Anonymous function:

```
quadgk(y -> func2(x, y, z), 1, 3)
```

Option 3 — `let` block (keeps variables local):

```
res, err = let x=5, z=3
    arg(y) = func2(x, y, z)
    quadgk(arg, 1, 3)
end
```

Inside another function:

```
function test_int(x, ymin, ymax)
    arg(y) = func3(x, y)
    return quadgk(arg, ymin, ymax)[1] # [1] extracts only the result
end
```

Multidimensional Integration

For multi-dimensional integrals, use dedicated packages:

- HCubature.jl
 - Cubature.jl
 - Cuba.jl
-

Lesson 13: Units of Measurement

Setup

```
using Pkg
Pkg.add("Unitful")
using Unitful
```

Attaching Units to Numbers

- Use the `u"unit"` notation:

```
one_meter = 1u"m"
one_meter = 1 * u"m"    # equivalent
```

Converting Units

- `uconvert(target_unit, value)` returns a new value in the desired unit without modifying the original:

```
uconvert(u"km", 1u"m")    # → 1//1000 km
```

Stripping Units

- `ustrip(unit, value)` converts to the given unit then removes the unit, returning a plain number:

```
ustrip(u"m", 1u"m")    # → 1 (Int64)
ustrip(1u"m")           # → 1 (no conversion needed)
```

Functions with Units

- Functions work naturally with unitful arguments—no special handling needed:

```
function compute_speed(Δx, Δt)
    return Δx / Δt
end
compute_speed(1u"km", 2u"s")    # → 0.5 km s^-1
```

- You can add type annotations using `Unitful` abstract types (`Unitful.Length`,

```
Unitful.Time, etc.):
```

```
function compute_speed(Δx::Unitful.Length, Δt::Unitful.Time)
    return uconvert(u"m/s", Δx / Δt)
end
```

Units in Structs

- **Do not** use abstract types in struct field definitions. Use `typeof` instead:

```
struct Person
    height::typeof(1.0u"m")
    mass::typeof(1.0u"kg")
end
```

Numerical Integration with Units

- Packages like `QuadGK` handle units automatically and return results with the correct derived unit:

```
using QuadGK
velocity(t::Unitful.Time) = 2u"m/s^2" * t + 1u"m/s"
quadgk(velocity, 0u"s", 3u"s")[1] # → 12.0 m
```

Key Takeaways

- `Unitful.jl` provides compile-time unit tracking with zero runtime overhead.
- Unit arithmetic (multiplication, division) automatically produces correct derived units.
- Use `uconvert` for conversion, `ustrip` to get a plain number.
- Prefer concrete unit types (via `typeof`) over abstract types in struct definitions.

Lesson 14: Interacting with Python

Core Package: `PyCall` is the bridge between Julia and Python. It lets you import and use Python libraries directly from Julia.

Two Setup Options

- **Automatic (simplest):** Let Julia handle everything via Miniconda. Best if you don't already use Python/Conda.

```
julia using Pkg Pkg.add("PyCall") using PyCall
```

- **Manual:** Best if you already have Anaconda/Miniconda installed and want to point Julia to your existing Python environment.

Manual Setup Steps

1. Install Miniconda (or use existing Anaconda).
2. Downgrade to Python 3.6 if needed: `conda install python=3.6 -y`
3. Configure environment variables and build PyCall:

```
julia ENV["PYTHON"] = "" ENV["CONDA_JL_HOME"] = "/path/to/miniconda3"
Pkg.add("Conda") Pkg.add("PyCall") Pkg.build("PyCall")
```

Importing Python Libraries

Use `pyimport` to bring in any Python module:

```
math = pyimport("math")
math.sin(3)    # returns 0.1411200080598672
```

PyPlot (matplotlib backend for Plots.jl)

Once PyCall works, you can install `PyPlot` to use matplotlib as a plotting backend:

```
Pkg.add("PyPlot")

using Plots
pyplot()    # switch backend

x = 1:0.1:2*pi
y = sin.(x)
plot(x, y, label="sin(x)")
```

Key Takeaways

- `PyCall` = Julia's interface to Python.
 - `pyimport("module_name")` is the equivalent of Python's `import`.
 - `PyPlot` provides a matplotlib-based backend for the `Plots.jl` ecosystem.
 - If you already use Conda, configure manually; otherwise, let Julia auto-install its own isolated Miniconda environment.
-

Lesson 15: Data Storage (HDF5)

What is JLD?

- `JLD.jl` is a Julia-specific dialect of the HDF5 file format, designed for storing and organizing large amounts of data on disk.
- It can save **variables, dictionaries, and custom structs**.

Saving Data

- Data is saved via a `Dict` mapping string keys to values, then written with `save()`:

```
using JLD
data_dict = Dict{"x" => x, "y" => y, "z" => z}
save("data_dict.jld", data_dict)
```

Loading Data

- Use `load()` to read a `.jld` file back into a dictionary:

```
data = load("data_dict.jld")
x = data["x"]
```

Storing Custom Structs

- Structs can be saved the same way (wrap in a `Dict`, call `save`).
- **Important:** The struct definition must exist in the workspace before loading the file. Otherwise, JLD reconstructs a generic type and you lose the named type (though field access still works).

```
# Define struct BEFORE loading
struct Person
    height::Float64
    weight::Float64
end
bob = load("bob.jld")["bob"] # returns Person(1.84, 74.0)
```

JLD2 (Evolution of JLD)

- `JLD2` is a pure-Julia reimplementaion with better support for native Julia types (e.g., `Unitful` quantities with units).
- Same API (`save/load`), but uses `.jld2` file extension.
- Generally faster and more flexible, but still under active development — prefer `JLD` if stability is critical.

```
using JLD2, Unitful
a = [1,2,3] * u"m"
save("data.jld2", Dict{"a" => a})
```

Key Limitations

- **JLD** (v1) **cannot** store data with units of measurement (`Unitful`). Use **JLD2** for that.
- Always define custom types before loading files that contain them.

Lesson 16: Parallel Computing

When to Use Parallel Computing

- Best for **independent operations**: mapping functions over arrays, Monte Carlo simulations, matrix operations.
- Rule of thumb: use **threads** for single-machine work; use **processes** for clusters.

Multi-Threading vs Multi-Processing

	Multi-Threading	Multi-Processing
Memory	Shared (lightweight)	Separate per process (heavier)
Sync issues	Yes — use locks	No
Scope	Single machine only	Can span a cluster
Startup cost	Low	High

Multi-Threading

Setup: Set thread count before launching Julia:

```
# Windows
set JULIA_NUM_THREADS=4
julia.exe

# Unix
export JULIA_NUM_THREADS=4
julia
```

Check available threads:

```
Threads.nthreads()
```

Parallel for loop with `@threads`:

```
using Threads
@threads for i in 1:length(x)
    results[i] = besselj1(x[i])
end
```

- Each iteration must be independent.
- Speedup $\approx$ number of cores.

Composable multi-threading with `@spawn` (Julia ≥ 1.3):

```
import Base.Threads.@spawn

a = @spawn slow_func(2)
b = @spawn slow_func(4)
res = fetch(a) + fetch(b)
```

- `@spawn` launches a task on an available thread.
 - `fetch()` retrieves the result.
 - **Only beneficial for slow functions** ($> 100 \mu\text{s}$). For fast functions, thread-spawning overhead dominates.
-

Multi-Processing

Setup:

```
using Distributed
addprocs(4)  # add up to your core count
```

Spawning tasks:

```
fetch(@spawn myid())      # runs on any available process
fetch(@spawnat 3 myid())  # runs on process 3 specifically
```

Defining code on all processes — @everywhere:

```
@everywhere using SpecialFunctions
@everywhere function my_func(x)
    return x^3 * cos(x)
end
```

Each process has its own memory, so functions/packages must be explicitly loaded on every process.

Distributed for loop (requires `SharedArrays`):

```
using SharedArrays
res = SharedArray(zeros(10))

@distributed for x in 1:10
    res[x] = my_func(x)
end
```

- Lightweight but **no load balancing** (splits iterations equally).

`pmap` — parallel map with load balancing:

```
pmap(my_func, 1:10)
```

- No `SharedArrays` needed; works on clusters.
- Assigns one value at a time per process by default (dynamic load balancing).
- Use `batch_size` when data transfer is expensive:

```
pmap(my_func, 1:1000, batch_size=100)
```


Key Takeaways

- **Threads** → single machine, lightweight, good for tasks > 100 μ s.
 - **Processes** → clusters or tasks > 100 ms; spawning is expensive.
 - Always **benchmark** (`@btime` from `BenchmarkTools`) to confirm parallelism actually helps.
 - If total computation < 100 ms, parallelism overhead likely isn't worth it.
-

Multiprocessing Module Tutorial

Core Concept

- Multiprocessing distributes independent computations across multiple CPU cores to speed up execution.
 - Best suited for **embarrassingly parallel** problems (e.g., applying a function to each element of an array independently).
-

Setting Up a Multiprocessing Module

Package creation (in REPL pkg mode):

```
] generate TestModule1
activate ./TestModule1
add Distributed SharedArrays ProgressMeter Interpolations
```

Module structure:

```
module TestModule1
  export func1, print_nprocs
  using Distributed
  using SharedArrays

  function func1()
    n = 200000
    arr = SharedArray{Float64}(n)
    @sync @distributed for i = 1:n
      arr[i] = i^2
    end
    return sum(arr)
  end
end
```

Calling the Module from a Script

```
using Distributed
addprocs(2) # number of CPU cores to use

@everywhere using Pkg
@everywhere Pkg.activate("./TestModule1")
@everywhere using TestModule1

func1()
```

Critical rule: Everything the workers need must be declared with `@everywhere` — environment activation, package imports, and module loading.

Key Macros & Functions

Tool	Purpose
<code>addprocs(n)</code>	Spawn <code>n</code> worker processes

<code>@everywhere</code>	Execute a statement on all workers
<code>@distributed for</code>	Parallelize a for-loop across workers
<code>@sync</code>	Block until all distributed tasks finish
<code>SharedArray{T}(n)</code>	Array shared across all workers (readable/writable by each)
<code>nworkers()</code>	Returns number of active workers

`@distributed` VS `pmap`

- `@distributed for` – batches work and assigns chunks to workers. Good when each iteration takes similar time.
- `pmap(f, collection)` – maps `f` over each element with **load balancing**. Better when execution time varies per element.
- Use `batch_size` argument to control how many items each worker processes before syncing back (higher = less data transfer overhead, less flexible balancing).

```
y = pmap(expensive_func, x, batch_size=10)
```

Sub-modules Inside a Parallel Module

```
include("SubModule.jl")
using .SubMod1

function func2()
    arr = SharedArray{Float64}(100)
    @sync @distributed for i = 1:100
        arr[i] = SubMod1.test_me(i)
    end
    return sum(arr)
end
```

Practical Tips

- Add `__precompile__()` at the top of your module to cache compiled code and speed up subsequent loads.
- Use `@showprogress` (from `ProgressMeter`) before `pmap` to display a progress bar.
- For reduction operations (e.g., summing), prefer `@distributed (+) for` over manually

filling a `SharedArray` and summing afterward.

- `@info "message"` is a cleaner alternative to `println` for logging.

Code Optimisation in Julia

Core Principle: Type Stability (Not Type Annotations)

- Julia's speed comes from **type stability**, not type annotations.
- **Type stability** means: the return type of a function must depend only on the types of its inputs, not on their values.
- Write code as **generic as possible**. Avoid unnecessary type annotations.
- Use type annotations only for **multiple dispatch** or when a function truly only works with one specific type.
- Prefer abstract types (e.g., `x::Real`) over concrete types when annotating numeric inputs.

Type Instability Examples & Fixes

Problem: Returning different types based on value:

```
function bad(x)
    if x > 0
        return x
    else
        return 0          # Int literal - breaks stability if x is Float64
    end
end
```

Fix: Use `zero(x)` to match the input type:

```
function good(x)
    if x > 0
        return x
    else
        return zero(x)    # returns 0 of the same type as x
    end
end
```

Problem: Changing a variable's type mid-function:

```
r = 0                # starts as Int
r += sin(i)          # becomes Float64 → type instability
```

Fix: Initialize with the correct type: `r = 0.0`

Detecting Type Instability

- Use `@code_warntype` to inspect compiled output.
- `Union{Float64, Int64}` or `::Any` in red = type instability.
- A single concrete type in blue = stable.

Profiling

- Use `@time` or `@btime` (from `BenchmarkTools`) for quick timing.
- For identifying bottlenecks line-by-line:
- **REPL:** `using Profile → @profile myfunc() → Profile.print()`
- **VSCode:** `@profview myfunc()` → opens a visual flame graph panel.

Performance Tips

Tip	Details
<code>@inbounds</code>	Skips bounds checking in loops when you're sure indices are safe. Can give ~3× speedup.
Static Arrays	Use <code>StaticArrays.jl</code> for small arrays (<100 elements) with known size. Huge speedups (up to 100×+ for matrix ops).
Column-major iteration	Julia stores matrices column-first . Iterate with the column index in the outer loop: <code>for j in cols, i in rows</code>
Avoid keyword args in hot loops	Positional args compile to less code. Use <code>foo(x, y=2)</code> over <code>foo(x; y=2)</code> in critical inner loops.
Avoid global variables	They hinder optimisation. Declare them <code>const</code> if they don't change, or pass them as function arguments.

Take-Home Message

Keep code **general** and **simple**. Focus on **type stability**. Speed comes from the compiler being able to infer return types, not from explicit type annotations.

Lesson 19: GPU Computations

What is ArrayFire?

- A library for array/matrix operations on GPUs.
- Supports **CUDA**, **OpenCL**, and **CPU** back-ends — code is portable across hardware.

Installation

1. Install the ArrayFire binary from arrayfire.com/download, then restart your system.
2. Add the Julia package:

```
using Pkg
Pkg.add("ArrayFire")
```

Core Concepts

CPU ↔ GPU Memory Transfer

- CPU and GPU do **not** share memory. Data must be explicitly moved.
- Transfers are slow — send data once, compute on GPU, retrieve only final results.

```
# CPU → GPU
host_to_device = AFArray(rand(100, 100))

# GPU → CPU
device_to_host = Array(host_to_device)
```

Generating data directly on the GPU (preferred over transferring):

```
a = rand(AFArray{Float64}, 100, 100)
```

This avoids the transfer overhead and uses ArrayFire's optimized GPU `rand`.

Operator Behavior on `AFArray`

- `^` is **element-wise** power (no broadcasting dot needed).
- `*` is **matrix multiplication**.
- Use `.*`, `.+`, etc. for explicit element-wise operations when needed.

Synchronization

- GPU operations are **asynchronous** by default.

- Call `sync()` before returning a result to ensure computation is complete.

Example: Monte Carlo π Estimation

```
function pi_gpu(n)
    return 4 *
        sum(rand(AFFArray{Float64}, n)^2 .+ rand(AFFArray{Float64}, n)^2 .<= 1) ./
            n
end
```

- All heavy computation stays on GPU; only the final scalar is transferred back.

Example: Mandelbrot Set

```
function mandelbrotGPU(c, maxIterations)
    z = c
    count = ones(AFFArray{Float32}, size(z))
    for n = 1:maxIterations
        z = z .* z .+ c
        count = count + (abs(z) <= 2)
    end
    return sync(log(count))
end

count = Array(mandelbrotGPU(AFFArray(c), maxIterations))
```

- Pattern: initialize on GPU $\rightarrow$ loop with element-wise ops $\rightarrow$ `sync` $\rightarrow$ transfer back with `Array()`.

Native CUDA in Julia

- Packages like `CuArrays.jl`, `CUDAnative.jl`, and `CUDAdrv.jl` allow writing **native CUDA kernels** in Julia.
- This solves the "**two-language problem**": Julia is both easy to prototype in and performant enough for production GPU code (no need to rewrite in C++).

Key Takeaways

Task	Function
Send data to GPU	<code>AFFArray(data)</code>
Retrieve data from GPU	<code>Array(gpu_data)</code>
Generate random data on GPU	<code>rand(AFFArray{Float64}, dims...)</code>

Synchronize async GPU ops

`sync(result)`

Lesson 20: Package Registration and Tests

Writing Tests in Julia

- Tests live in `test/runtests.jl` inside your package directory.
- Use the `Test` standard library package.
- A test is an expression that evaluates to `true` (pass) or `false` (fail).

Core macros:

```
using MyPackage, Test

@testset "descriptive name" begin
    @test some_expression == expected_value
    @test isapprox(computed, expected, atol=tolerance)
    @test_throws ExceptionType my_func(bad_arg)
end
```

- `@test` — checks a single boolean expression.
- `@testset` — groups related tests under a named block.
- `@test_throws ExceptionType expr` — verifies that an expression throws a specific exception.
- `isapprox(a, b, atol=tol)` — useful for floating-point comparisons.

Running tests: In the REPL, type `] test` (package mode).

Continuous Integration with Travis CI

- Travis runs your tests automatically on every push to GitHub.
- Requires a `.travis.yml` file at the root of your repo.

Minimal `.travis.yml` structure:

```
language: julia
os:
  - linux
  - windows
  - osx
julia:
  - 1.0
  - nightly
matrix:
  allow_failures:
```

```
- julia: nightly
after_success:
  # submit coverage to Coveralls / Codecov
```

- You can test across multiple Julia versions and OSes.
 - `allow_failures` lets nightly builds fail without marking the whole build as failed.
-

Code Coverage (Coveralls / Codecov)

- These services report what percentage of your code is exercised by tests.
 - Aim for >90% coverage before registering.
 - Coverage results are submitted automatically via the `after_success` block in `.travis.yml`.
 - Add badges to your `README.md` to display build status and coverage.
-

Package Configuration (`Project.toml`)

Before registration, ensure `Project.toml` has:

- `name`, `uuid`, `authors`, `version` fields.
- First release version: `0.1.0` (beta) or `1.0.0` (mature).
- A `[compat]` section specifying dependency version bounds (**mandatory** for registration):

```
[compat]
SomeDep = "2.0.0"
julia = "^1.0.0"
```

- `^1.0.0` means compatible with all 1.x releases (up to but not including 2.0).
-

Registering a Package

1. Host your package on a **public GitHub repository**.
 2. Install the [Julia Registrator GitHub App](#) and grant it access to your repo.
 3. Optionally install [TagBot](#) for automatic tagging/releases.
 4. Go to pkg.julialang.org/registrator, enter your repo URL and branch, then submit.
 5. A pull request is opened on the Julia General registry; if checks pass, your package is merged.
-

Versioning (Semver Conventions)

Change type	Version bump	Example
Bug fix, no interface change	patch	0.1.0 → 0.1.1
New features, backward-compatible	minor	0.1.x → 0.2.0
Breaking changes / new interface	major	0.x.y → 1.0.0

- Never skip versions (e.g., don't jump from 0.1.0 to 0.3.0).
- After bumping the version in `Project.toml`, re-submit via the Registrator.

Automation Tip

- `PkgTemplates.jl` can scaffold a new package with Travis, Codecov, and Coveralls integration pre-configured.

Lesson 21: Interoperability

Julia can call code from several other languages directly. Here's how:

Python (via `PyCall` / `PyJulia`)

- **Julia → Python:** Use `PyCall.jl` (covered in an earlier lesson).
- **Python → Julia:** Use `PyJulia`.
- Install: `pip install julia`
- Setup in Python:

```
python import julia julia.install()
```
- If Julia isn't on PATH, set an environment variable `julia` pointing to the executable.
- Call Julia from Python:

```
python from julia import Base Base.sind(90)
```

C++ (via `Cxx.jl`)

- Install: `Pkg.add("Cxx")`
- Use the `cxx"""..."""` string macro to include headers and define C++ functions inline.
- Wrap a C++ function as a Julia function with `@cxx`:

```
julia using Cxx cxx""" #include <iostream> """ cxx""" void mycppfunction() {  
std::cout << "Hello" << std::endl; } """ julia_function() = @cxx
```

```
mycppfunction() julia_function() # prints "Hello"
```

Wolfram Mathematica (via `MathLink.jl`)

- Requires Mathematica (paid) or Wolfram Engine (free).
- Set environment variables `JULIA_MATHKERNEL` and `JULIA_MATHLINK` to the correct paths.
- Install: `Pkg.add("MathLink")`
- Use the `W"..."` string macro to write Wolfram expressions, and `weval()` to evaluate them:

```
julia using MathLink weval(W"Sin"(1.0)) # returns 0.8414...
```

MATLAB (via `MATLAB.jl`)

- Install: `Pkg.add("MATLAB")` (platform-specific setup required — see docs).
- Set `MATLAB_HOME` env var to point to a specific MATLAB installation if needed.
- Create MATLAB arrays and convert them to Julia:

```
julia using MATLAB x = mxarray(Float64, 42) # 42x1 MATLAB zero array j =  
jarray(x) # convert to Julia array
```

R (via `RCall.jl`)

- Install: `Pkg.add("RCall")` — auto-installs R via Conda if not found.
- Access the R REPL from Julia by typing `$` after `using RCall`.
- Transfer data between languages:
- `@rput var` — sends a Julia variable to R.
- `@rget var` — retrieves an R variable into Julia.

```
julia using RCall z = 1 @rput z # z is now available in R @rget r # brings R  
variable `r` into Julia
```

FORTRAN (via `ccall`)

- No dedicated package needed; use Julia's built-in `ccall` to call compiled FORTRAN libraries.
- Example calling BLAS `ddot_`:

```
julia product = ccall(:ddot_, "libLAPACK", Float64, (Ref{Int32},  
Ptr{Float64}, Ref{Int32}, Ptr{Float64}, Ref{Int32}), n, DX, incx, DY, incy)
```

General Tip

- The [JuliaInterop](#) GitHub organization hosts many more integration packages for other languages and frameworks.

Lesson 22: Multiple Dispatch

Type Annotations

- Use `::` to annotate parameter types: `function f(x::Number)`
- Julia is **optionally typed** — annotations are not mandatory and generally not needed for performance (as long as code is type-stable).
- Annotations improve readability and can make debugging easier by giving clearer error messages.
- **Prefer abstract types** (e.g., `Number`) over concrete types (e.g., `Float64`) to keep functions general and extensible.
- Rule of thumb: don't add type annotations unless they are general or needed for a specific reason (like multiple dispatch).

Multiple Dispatch

- Multiple dispatch = a function behaves differently based on the **number and types** of its arguments.
- A "function" is the generic name; each specific implementation is called a **method**.
- Define multiple methods for the same function name with different type signatures:

```
function f(x::Number)
    return x^2 + 42
end

function f(x::String)
    println("Hello! I see you like $x")
end

function f(x) # fallback for any other type
    println("type of x: $(typeof(x))")
end
```

- Dispatch also works on the **number of arguments**:

```
function f(x, y)
    return x + 3*y
end
```

- Broadcasting works naturally: `f.([1,2,3])` applies the function element-wise.

Inspecting Methods

- `methods(fname)` — lists all methods defined for a function.
- `@which f(2)` — shows which specific method would be called for a given argument.

Extending Existing Functions

- You can add new methods to functions from other libraries (including `Base`).
- You must `import` the function first:

```
import Base.+  
+(x::String, y::String) = "$x$y"
```

- This also works inside modules — importing the module brings the extended method into scope:

```
module TestMe  
    import Base.+  
    +(x::String, y::String) = "$x$y"  
end  
  
using .TestMe  
"Techy" + "Tok" # → "TechyTok"
```

Key Takeaway

Multiple dispatch is central to Julia's design. It lets you keep function names concise while providing specialized behavior per type combination, enabling both flexibility and performance.